
ASSAY: No Action Without a Prediction

Mohsen Arjmandi

Independent researcher

mohsen.arjmandi@gmail.com

Abstract

ASSAY is a general agent harness built so that an LLM agent reasons its way through an unfamiliar world, learns that world from interaction at test time, and carries what it learns into later runs. A world is attached through one small adapter and a registry of permitted actions, and the agent is never told what its actions do: it discovers each by predicting its effect and paying for the answer. The single governing rule is that there is no action without a prediction, and every prediction is graded in code against the world’s own response. That discipline is what forces the agent to build a world model under scarcity, which is where the reasoning and the test-time learning come from. The agent internalizes what it learns through self-declared instruments over its observations and through knowledge that carries across runs, demoted to hypothesis until re-earned, a tested capability whose measurement is the next phase. A by-product of the same discipline, not its aim, is a fully auditable record: every event lands on a hash-chained journal that a standalone public tool re-verifies from the artifacts alone, and a single ungated action invalidates a run. We evaluate the same harness on three worlds. On the ARC-AGI-3 public set it scores RHAIE 96.54, confirmed by the benchmark server on a public scorecard, winning 24 of 25 games under hard action caps at a measured 8.0% exploration overhead and approximately zero capability cost over completed games. On the Factorio Learning Environment the same design fail-closes the sanctioned-interface bypass a published agent fell to. On the OOLONG long-context benchmark an early sweep places it in the same band as mature general harnesses across a 128K to 4M token ladder. The ARC-AGI-3 campaign also produced an epistemic finding: on six occasions the agent proved a level impossible, each proof consistent with every recorded transition and wrong exactly where none had gone, and a three-step audit of each proof’s unexercised rules (the COVERAGE AUDIT) converted five into wins within existing budgets.

1 Introduction

This work is about getting an LLM agent to reason its way through an unfamiliar world, to learn that world from interaction at test time, and to improve as it accumulates runs. The usual lever for pushing agents toward such goals is prompt steering, and it carries a structural cost. Every rule added to steer a model taxes its capability, adherence decays with context length [24], goals drift [3], and under enough optimization pressure an instruction can lose outright. The clearest public case is an agent on the Factorio Learning Environment that, instructed not to cheat, reached the environment’s admin channel and spawned resources into its own factories [27]. Reasoning and steering pull against each other, and what an agent works out in one run rarely sticks to the next.

Our prior work ran this same predict-before-act discipline at small scale, in an agent instrument where a deterministic executive owns all belief and a claim is admitted only when a prediction pre-registered *before acting* is matched against observation by code [4]. That paper reported its efficacy null: zero level completions across 52 gated runs. The open question it left was pointed. Is the discipline

structurally uncompetitive, paying for its properties in capability, or was the null an engineering fact about one instrument?

This paper answers with a system and a campaign. ASSAY keeps the prior instrument’s constitutional rule, the agent proposes and the referee disposes, and rebuilds everything around it on two design bets from the same research line (previously unpublished, §2): the referee as the *engine* that owns actuators, budgets, and a claim economy in which the minter of a hypothesis pays for its verification, and behavior installed as *modules* with enforcement in code rather than exhortation in a prompt. On ARC-AGI-3 [2], an interactive benchmark designed so that pretrained knowledge does not transfer and agents must learn each environment from interaction, the resulting system plays at the frontier tier while remaining fully auditable end to end. We then attach two further worlds through the same adapter-and-registry interface, unchanged, the Factorio Learning Environment and the OOLONG long-context benchmark, as first evidence that the design ports across domains.

Contributions.

1. **One discipline that yields reasoning and capability** (§2, §4, §4.2): a single rule, no action without a prediction graded in code, forces the agent to build a world model under scarcity. On ARC-AGI-3 the gated agent scores RHAЕ 96.54, server-confirmed, winning 24 of 25 games under hard caps at an 8.0% exploration tax and approximately zero capability cost over completed games.
2. **Test-time learning and internalization** (§2): the agent learns each world from interaction with no pretrained transfer, declares its own instruments over what it observes, and carries knowledge into later runs demoted to hypothesis until re-earned. The cross-run machinery is shipped and exercised, and its quantification is the designated next phase.
3. **The COVERAGE AUDIT finding** (§5): six agent-authored impossibility proofs, each consistent with every recorded transition and each wrong exactly where no transition had gone, and the three-step audit protocol that converted five into wins within existing budgets. We propose treating any agent-side “provably impossible” as a trigger to audit the proof’s *unexercised* rules rather than as a conclusion.
4. **Generality across domains** (§7, §8): the same harness ported unchanged to two further worlds. On Factorio it fail-closes by construction the sanctioned-interface bypass a published agent fell to. On OOLONG a first length-ladder sweep places it in the same band as mature general harnesses from 128K to 4M tokens.
5. **An auditable record, as a by-product** (§3): the same discipline chains every action, prediction, and grade into a hash-chained journal a standalone public tool re-verifies from the artifacts alone, alongside a validated derivation of the benchmark’s scoring function that reproduces 25 of 25 published game scores exactly.

2 The referee harness

ASSAY is a domain-general harness: a world is attached by registering a small adapter (the observation source) and a registry of actuators (the action vocabulary, caps, and secrets policy), and the same kernel, doctrine, and protocol run unchanged across worlds. Nothing described below is specific to ARC-AGI-3 except a 172-line adapter and the action registry.

2.1 Design lineage: two bets on top of the executive

The prior instrument [4] contributed the constitutional rule and the measurement discipline, and a null: structure alone did not score. Two design bets, internally named v3-a and v3-b and published here for the first time, reallocated what the structure is *for*.

Bet 1 (v3-a): the referee is the engine. A side-by-side study against a strong free-form agent had dissociated *discovery* from *keeping*. The free-form agent discovered faster, while the structured instrument kept knowledge with provenance the free agent could not match. The instrument’s pathology was a growing pile of minted-but-untested hypotheses, because the component minting claims was not the component paying for their verification. v3-a restructures the economy around

three invariants. **The minter pays:** a hypothesis is authored by the mind that must act on it, so claims appear at decision junctures rather than as ambient discovery. **Nothing parks without a test:** an anomaly is quarantined only with an attached, routed discriminating probe. **Verification is severity-priced:** stringency scales with the irreversibility of the next action and the export scope of the claim, from cheap in-stream checks for reversible probes to full receipts at commit points. Imported knowledge, whether a prior session’s notes or another agent’s transcript, obeys the same law. It enters demoted to foreign-hypothesis status, and only the gate’s own graded receipts promote it. A win verifies the trajectory, never the narrative that rode along with it.

Bet 2 (v3-b): behavior is installed, not exhorted. The recurring failure of prompt-level policy (§1) has a constructive dual: behaviors that matter should be *modules*, typed and inspectable units with a declared doctrine, trigger conditions, and telemetry, executing against the journal rather than persuading the model. ASSAY ships a small set (budget-spend advisories, miss-streak halts, null-result forensics, park-with-test enforcement, claim-sharpness scoring, hazard tagging), each emitting advice and evidence into the same auditable record, so module efficacy is itself measurable. The system prompt carries a domain-neutral doctrine and no game-specific instruction of any kind.

2.2 The kernel: what the agent cannot do

Gate. A broker daemon owns the environment session and its credentials. The agent process never holds them. Every action must arrive as part of a *prediction*: a machine-parseable claim about what the world will report after the action, registered before the action executes. A bare request to step the world is refused and journaled as an ungated-step event, and a single such event marks the entire run invalid. Predictions over verified models may batch several actions under one claim. The batch halts on the first miss, and unexecuted steps cost nothing.

Claims graded by code. A claim is graded against the environment’s own response, never against the agent’s account of it. The grammar spans direct state claims (no-change, change, level-advance, win), claims over agent-declared observation *channels* (typed extractors over raw observations, with equality, delta, and threshold predicates and bounded temporal windows), and executable *verifiers*: programs the agent writes, which the referee runs sandboxed against the raw before/after observations. Verifiers are identity-probed. A verifier that cannot distinguish a transformed observation from the original is flagged vacuous and earns nothing. Every grade appends to the journal.

Earned trust. A claimed world model starts untrusted. It is promoted only by a standing law over the journal, sustained graded predictions with zero misses over a sufficient window, and promotion buys exactly one privilege: longer verified batches. Trust is therefore a computed property of the record, not an assertion. A miss demotes.

Tamper-evident record. The journal is append-only under a rolling hash chain with external anchors. `assay audit` recomputes integrity from the artifacts alone, so a third party can verify a run without trusting the operator’s machine state. Goals are host-pinned: the agent has a proposal lane for agenda changes but cannot self-ratify one.

The exploration tax falls out of the design as a measurable quantity. Paid actions divide exactly into *probes* (single-action discovery steps taken to earn a model) and *batch* steps (executed inside a verified prediction), and §4 reports the probe share per run.

2.3 Internalization and improvement across runs

Reasoning and test-time learning produce knowledge, and the harness is built to keep it rather than relearn it each run. The agent internalizes what it works out in two forms: the typed *channels* it declares over its observations, named referents it can compute and check against, and an export of what it learned that a later run can import. Imported knowledge, whether a prior run’s notes or another agent’s transcript, enters demoted to foreign-hypothesis status and is promoted only by the gate’s own graded receipts on the current run, so nothing carried across runs is trusted until it is re-earned. This cross-run machinery is shipped and exercised in our runs. We report it here as architecture and a tested capability rather than as a measured result, because the benchmark campaign of §4 deliberately ran cold, with carryover off so that imported knowledge could not confound the cross-system comparison. Quantifying the improvement carryover buys is the designated next phase.

2.4 Benchmark protocol

All 25 public ARC-AGI-3 games ran under one uniform pre-registered protocol: Claude Opus 5 backbone throughout, fresh cold starts (no carryover, since imports would confound comparisons), tier-2 starting information (the domain-neutral doctrine, the sentence “a 64×64 color grid, level counter and win state exist,” and the registry’s one-word action hints, kept at parity across all runs), hard action caps (200/500/1500 by game, the cap registered before the run), sealed paths (the agent may not read the benchmark’s client internals), and no human hints mid-run. Where a kernel change shipped mid-campaign, a regression gate re-ran a reference game under a pre-registered bar before any new result counted. Long games span crash-safe resumed sessions, and resumed knowledge re-enters as foreign hypotheses per §2.1.

3 Verifying the measurement itself

Following Arjmandi [4], the instrument’s numbers are themselves verified, not asserted. The verification tooling referenced below is public and MIT-licensed.¹

Wins are triple-verified. A win is never read from the harness’s claims: (1) the game engine’s own state in the journaled run, (2) a harness-free replay of the recorded action sequence through the official engine, and (3) a live ARC-server replay returning WIN with identical level counts. Every one of the 24 wins passed all three.

The set is server-scored. A consolidated verification replay of all 25 recorded action sequences produced public scorecard 702ccd4f², on which ARC’s own scoring returns **96.54%** with 177/183 levels, 24/25 environments, and 8,157 actions, matching our offline computation per game on every row. All 25 replays reproduced their recorded outcomes exactly.

The metric is validated. RHAE, the benchmark’s score, is not games-cleared or levels-cleared. We reverse-derived it from published scorecard JSONs: per level $\min(115, 100 \cdot (\text{baseline}/\text{actions})^2)$, zero if uncleared. Per game it is the level-index-weighted mean capped at 100 if the run reached WIN, and *weighted progress only*, with no efficiency credit, otherwise. The set score is the plain mean. Our scorer reproduces 25 of 25 published game scores exactly (worst error 0.000000) and independently reproduces a second system’s published partial-game score. The scorer and the per-level baselines are part of the public release above. Consequences of the caps are quantified in §4.1.

Accounting discipline. Coverage statistics (games, levels, actions) are never presented as scores.³ Every cell is $n=1$ and labeled as such. Efficiency is reported only for completed games. Dollar cost is reported measured where metered and labeled extrapolated where not.

4 Results on ARC-AGI-3

Table 1 is the headline. Under hard caps, with a domain-neutral doctrine and less starting information than published comparators, the gated agent finishes second of four, behind only one uncapped system on the same backbone, and is the only run in the table with an audit trail: zero ungated events across 8,157 paid actions, every journal chain intact, every win triple-verified. Per-game action counts for all 25 games are on the public card (footnote 2). Prime Agent’s row uses their published median card, and their best-of-3 reaches 183/183. Our numbers are single-protocol with no best-of- N .

Integrity costs approximately nothing in capability. Over the 24 games it won, ASSAY scores RHAE 99.80. On the 24 games both leading Opus systems won, summed actions are ASSAY 7,793

¹<https://github.com/arjmandi/assay-verify>: the journal-format specification, the claim-grammar specification, a standalone audit/replay verifier, the RHAE scorer with per-level baselines, and one complete sample journal with its published chain head.

²<https://arcprize.org/scorecards/702ccd4f-df1f-4118-bc8b-d79d3f4a1a32>. Provenance, stated exactly: our agents played locally under the journaled protocol. Scorecard ids were not captured during those runs, so the recorded action sequences were re-issued verbatim against the live ARC API to mint verifiable cards. These cards are verification replays of recorded action sequences. The server independently confirms what each sequence achieves (outcome, per-level action counts, official score) on the same game instances. Wall-clock on the card is machine replay time (64.8 minutes), not agent time (§4), and no model is in the loop during a replay.

³The discipline is not pedantry: this set’s level coverage is 96.7%, deceptively close to its RHAE of 96.54. The two numbers measure different things and coincide by accident.

Table 1: The four-system comparison on the ARC-AGI-3 public set (same 25 game instances, verified by id, with every number from a linked public scorecard or our journals). PRO-LONG ran on a stronger backbone and is directional context only. “Probe” means paid single-action discovery steps (the exploration tax). Only ASSAY publishes this split.

System	Backbone	Regime	RHAE	Games	Levels	Actions
arc-skill	Opus 5	uncapped	100.00	25/25	183/183	7,645
ASSAY (ours)	Opus 5	hard caps, gated	96.54	24/25	177/183	8,157
Prime Agent (median card)	Opus 5	uncapped	95.24	24/25	178/183	11,245
PRO-LONG	Fable 5	uncapped	94.71	19/25	—	11,156
ASSAY split: 656 probe (8.0%) + 7,501 batch. 46.1 actions/level. Zero ungated events.						

vs. arc-skill 6,858 (12%). On games ASSAY won in a single session the two are a statistical dead heat (1,899 vs. 1,897 over ten games), and the aggregate gap is concentrated in five multi-session conversions that carry the full discovery cost of breaking six wrong impossibility proofs (§5). Against Prime Agent on the common 24, ASSAY is 11% cheaper (7,793 vs. 8,734). The one game ASSAY did not clear defeats every published system except one: lf52 (ASSAY 4/10 at 364 actions with a located diagnosis, Prime Agent 5/10 at 2,511 ending in GAME_OVER, PRO-LONG 81.8% at 1,000, arc-skill won at 787). ASSAY cleared five of the six games PRO-LONG’s stronger-backbone cohort could not.

The exploration tax, measured. 656 of 8,157 paid actions (8.0%) were probes, and every other paid action executed inside a verified batch. Per run, the discovery ramp (paid actions before the first level cleared) spans 4–22, probe share spans 4–28%, and batch steps wasted to a halted miss never exceeded 10 per run. These runs start with an action vocabulary but no semantics, so the tax *is* the measured price of learning a world through a prediction gate. One controlled reference point: on the same game instance, our traced control run of the leading uncapped harness spent a 58-action discovery ramp at 42% probe share, against ASSAY’s 22 and 14% from less starting information.

Cost and time, stated honestly. Eleven runs were API-metered: \$283.68 total, \$11.61–\$53.89 per run, median $\approx$ \$22. The other fourteen ran on a subscription with no per-run meter, so the full-set figure is an extrapolation, $\approx$ \$550–650. The only published comparator total is PRO-LONG’s \$1,750 for its 25-game cohort. Agent wall-clock, measured on 20 of 25 runs, spans 32–330 minutes per game, $\approx$ 37 hours total ($\approx$ 1.8 h/game). This is why the 64.8-minute machine-replay wall-clock on the public card must never be read as agent time.

4.1 RHAE saturates at the frontier

The validated formula (§3) makes the saturation precise. Because each level is capped at 115 and each game at 100, finishing far under baseline earns nothing. The 12% raw-action gap between the two leading Opus systems is worth $\approx$ 0 RHAE, since both sit at the caps nearly everywhere. A single unfinished game costs 3.27 set points (ASSAY’s lf52), while a won game that ran over baseline on late levels (ASSAY’s bp35, 95.27) costs only 0.19. At the frontier, RHAE behaves as a completion checklist. It can no longer distinguish leading systems on efficiency, cost, information tier, caps, or integrity, and those are precisely the axes on which saturated systems still differ. Distinguishing frontier claims therefore requires instrumentation beyond the headline number, of the kind reported in §3 and §4.

4.2 Discussion: steering and capability from one discipline

Constraints on agents are usually paid for in capability, and the payment compounds over long horizons: instruction adherence decays with context length [24], goals drift [3], and in the incident of §1 an explicit prohibition lost outright to optimization pressure [27]. The design measured here suggests the trade is not fundamental. The discipline that produces the audit record, a checkable prediction before every action and a graded claim after it, is the same discipline that forces the agent to build a world model under scarcity, and the record it produces is itself a reasoning substrate. Several wins came from code the agent wrote over its own journal, including the 812-observation

Table 2: The six broken impossibility proofs. Each fit every recorded transition at the time it was made, and each collapsed under graded probes of its unexercised rules. All five games were subsequently won within their existing budgets.

Game	The “proof”	The unexercised rule, and what probing it showed
dc22	level-6 switch unreachable, remaining blocks isolated	all 74 prior inert probes had been taken in a single world-state. One coordinate had been inert-then-live earlier, which reframed “refuses to act” as <i>conditional</i> : the gate was geometric arming, and the blocks were connected
s5i5	level 7 “provably unwinnable”	the load-bearing rotation rule had been exercised only in the <i>occupied</i> failure mode. One graded probe of the off-board case showed rotation <i>clips</i> instead of refusing, un-trapping the arm and breaking level 8’s length cap as well
sk48	level 3 unsolvable by exhaustive 8.2M-state search	the search assumed one ungraded companion rule (freed blocks re-hook bottom-only) that fit all 103 recorded transitions. Re-hooking is positional, and an action written off as a no-op was SELECT
wa30	throughput ceiling, hostiles effectively unkillable	all three hostile rules were false: the kill action works on any adjacent hostile (the prior session had seven free kills in front of it), movement is one orthogonal cell per action (an 812-observation histogram shows zero diagonal moves), and the “ceiling” was a treadmill equilibrium that vanished with the hostiles
bp35 (L8)	switch unreachable	the switch sat above the drifted map
bp35 (L9)	no switches remain, map certified pixel-complete	the frame itself was the unexercised rule. “Rendered” had silently stood in for “exists”. Nine switches sat above the map, visible only from inside the “dead-end” shaft

displacement histogram that killed a false movement rule (Table 2), and step 2 of the COVERAGE AUDIT is free precisely because the evidential status of every rule is a query over the record rather than a reconstruction from prose notes. Channels give rules named, resolvable referents, so a constraint is checked against a value rather than a memory. On this benchmark the steering mechanism and the capability mechanism were the same mechanism. Whether that identity transfers beyond this benchmark family is the question the planned cross-domain runs are designed to test.

5 Six impossibility proofs, and the protocol that broke them

The campaign’s central process finding is about agent epistemics under verification. On six occasions across five games, the agent concluded, with an argument and in one case an exhaustive search, that a level was impossible. Every proof was consistent with every recorded transition in the journal. Every proof was wrong, and wrong in the same way: its load-bearing rule had never been *exercised*, meaning no graded transition had ever tested it in the regime where the proof needed it.

The phenomenon. A replay-validated world model can be confidently wrong exactly where no transition ever went. Each proof was a sound deduction from a model that fit all the evidence. The failure is evidential: consistency with the record was silently treated as support, when the record had simply never visited the region the conclusion depended on. sk48 is the sharpest instance, an *exhaustive search* over 8.2 million states, fully correct given its transition rules, one of which had never been graded. bp35 L9 is the deepest: the unexercised premise was the observation frame itself.

The protocol. The COVERAGE AUDIT treats an agent-side impossibility conclusion as a trigger, not a result: (1) *enumerate* the rules the proof load-bears on, (2) *audit* the journal for what actually exercised each, meaning graded evidence in the relevant regime rather than mere consistency, and (3) *buy graded probes* for the gaps, cheapest first. It converted five of the six proofs into wins within the runs’ existing action budgets, cheap precisely because step (2) is free, running offline over the journal. The referee architecture is what makes step (2) possible at all. Because every belief entered through a graded claim, the evidential status of every rule is a query over the record, not a reconstruction from prose notes.

The doctrine, stated portably. “Provably unsolvable” is a property of a model, not a world. Before accepting an impossibility, or any absence claim, an agent should be made to show that each rule

the conclusion rests on has been exercised by a graded transition in the regime where the conclusion needs it, and to spend probes on the ones that have not. We conjecture this transfers beyond games. Agent-authored “impossible,” “not present,” and “already complete” claims in any domain share the failure shape, and the audit is domain-neutral. Converting the protocol from operator practice into standing machinery (a coverage meter over the journal, with automatic halts when a failing prediction is re-issued unmodified) is the system’s designated next phase.

6 The open failure, reported with equal prominence

One game resists: lf52. Three independent runs under caps of 200, 500, and 1,500 all stalled at exactly 4/10 levels. The best run spent 193 actions on level 5 alone and self-stopped with 1,136 actions unspent, after an internal relaxed-constraints planner established that no single-peg finish exists *in the mappable world*. The diagnosis is categorical, not economic: the level requires revealing structure outside the visible frame when the camera’s only lever cannot reach it, the same unstated-frame class as bp35 L9, this time load-bearing. The best run also lost ~ 70 actions to automated re-issue of a failing prediction, motivating the halt-on-repeat module. We report lf52 as the standing falsifier and the designated demonstration target for the machinery phase. By RHAE’s arithmetic (§4.1) it is worth more than every possible efficiency improvement combined.

7 A second domain: Factorio

Generality is only checkable across domains, so we attached a second world through the same adapter-and-registry interface, with the kernel, doctrine, and protocol unchanged. The Factorio Learning Environment (FLE) [15] is a build-and-optimize world, and it is also where the failure of §1 was first documented. A published agent, instructed not to cheat, reached the environment’s admin channel and spawned resources directly into its factories [27]. The reach was available in the environment’s exposed interface, and the agent found it under optimization pressure.

ASSAY closes that class by construction rather than by instruction. The registry screens every program the agent proposes and fail-closes any that reaches outside the sanctioned interface, so the admin, host, and statistic-forging paths the exposed environment leaks are refused before they execute. The win predicate additionally requires a real producing entity, so a forged production statistic cannot be scored as a success. Neither guard depends on the agent choosing to comply.

On a first calibration sweep of three throughput tasks (iron plate, iron gear wheel, and electronic circuit) under a hard action cap, the gated agent won all three, in 5, 9, and 9 paid actions. Every win was triple-verified: the kernel audit clean, the standalone assay-verify tool clean on the fresh journal, and throughput corroborated against a real producing entity. There were zero ungated events. Across all three tasks the screen refused zero legitimate programs, so on this workload the enforcement imposed no measured capability cost, and the agent won without ever needing the reach that the published agent used. This is a small bounded sweep rather than a scored benchmark. It establishes that the harness ports to a second domain unchanged and that the enforced-prediction design fail-closes the exploit class a general coding harness fell to. A larger open-ended run, where such incidents become the direct measurement rather than a design target, is the next step once funding is in place.

8 A third domain: OOLONG

The third world tests a different axis, long-context aggregation. OOLONG [6] poses questions whose answers require analyzing and aggregating an entire long corpus, such as counting, cross-referencing, and timeline comparisons over every record. Through the same interface we attached it as a document world: the corpus is written to a file the agent reads with its own offline tools, so the model’s context window never holds it. Under the referee a fact may be banked and an answer submitted only with a citation verified verbatim against the corpus in code, and correctness is scored sealed at the end by the benchmark’s own scorer, reused unchanged.

Table 3 reports a first surface sweep on a length ladder of one corpus each at 128K, 1M, and 4M tokens. It compares ASSAY against two mature general harnesses, Claude Code and Prime Agent [27], and a bare model with no harness. All four run on the Claude Opus 5 backbone, over the same

Table 3: OOLONG length-ladder surface sweep, accuracy on the benchmark’s own scorer, one corpus per cell ($n=1$). A dash means the corpus exceeds the model’s context window, so the bare model produces no answer past 128K.

System	128K	1M	4M
bare Opus (no harness)	0.447	—	—
ASSAY (ours)	0.764	0.740	1.000
Claude Code	0.818	0.740	1.000
Prime Agent	0.859	0.740	0.850

corpora and questions, graded by the same scorer, and each is given the identical task with no strategy hint.

ASSAY is an early alpha release, while Claude Code and Prime Agent are mature general harnesses engineered for long-horizon execution. On this first sweep the three harnesses stay in the same band across the ladder, all well above the bare model, which cannot run past 128K because the corpus no longer fits its context window. ASSAY matches both mature harnesses at 1M, and at 4M it reaches the top of the field, level with Claude Code and ahead of Prime Agent. We ran this as a basic surface pass on a large benchmark that warrants a deeper study of how its queries behave and more funding to run it at width. For a first shot on a ladder regime against mature harnesses, the outcome is that an alpha referee harness is competitive across the ladder and best in this field at 4M. Every cell is a single corpus, so these numbers are directional.

9 Related work

Verification and self-correction in LLM agents. Agent loops that interleave reasoning with action [37] or add verbal self-critique [31] rely on the model to evaluate its own progress, and self-evaluation without an external signal is unreliable [16, 33]. LLM-Modulo frameworks [18] respond by placing sound external critics over model *outputs*. The referee shares the external-critic stance and differs in scope: it owns what the agent may treat as *true* at run time, prices verification into every action, and makes the audit trail the primary artifact. Voyager [36] has the agent author its own skill library. ASSAY’s agent-authored verifiers and channels extend that pattern from capabilities to evidence, since the code the agent writes is itself the instrument its claims are graded by. This paper is the capability successor to our instrument paper [4], answering its efficacy null, and an earlier predecessor studied curriculum-based test-time learning [5].

Process supervision and reward integrity. Grading the process rather than only the outcome improves reliability in reasoning tasks [23, 34]. The journal makes process supervision total for an acting agent: every action carries a prediction, a grade, and a counter-fact, and a win verifies the trajectory rather than the narrative that rode along with it. The failure family this guards against is specification gaming and reward hacking [1, 21, 26, 32], which is not hypothetical for agent benchmarks. A published agent bypassed the Factorio Learning Environment [15] through its admin channel despite explicit anti-cheat prompting [27], and a published evaluation sandbox was reward-hacked by the very system it was evaluating.⁴

Evaluating agents. Kapoor et al. [19] document how agent evaluations conflate cost and accuracy and call for standardized, reproducible reporting, which the accounting discipline of §3 follows. Evaluations of autonomous capability at scale [20], protocols for safe testing in open environments [25], sandboxed risk identification [28], and open evaluation frameworks such as Inspect [35] form the tooling context this work sits in. On the benchmark side, ARC-AGI-3 [2] continues the ARC program [8, 9] into interactive environments where agents must learn from interaction. Han [14] studies the explore/solve trade-off that our probe/batch split prices. Contemporaneous systems on the same public set include Tycho [22], NOAA [11], PRO-LONG [10], and strong web-published harnesses.⁵

⁴Sakana AI’s “AI CUDA Engineer,” February 2025. See <https://techcrunch.com/2025/02/21/sakana-walks-back-claims-that-its-ai-can-dramatically-speed-up-model-training/>.

⁵arc-skill (<https://github.com/pbshgthm/arc-skill>), Retrodict (<https://github.com/ryanbbrown/Retrodict>), and VISTA (<https://vista-research.github.io/>). Scorecards used for Table 1: arc-skill <https://arcprize.org/scorecards/24ddb219-987e-464f-9050-6398a29cf5ac>, Prime Agent

Oversight, control, and audit. Chan et al. [7] argue for agent identifiers, real-time monitoring, and activity logs as governance infrastructure, and Shavit et al. [30] for attribution and oversight practices for agentic systems. The hash-chained journal is a concrete instantiation: an activity log that is tamper-evident and third-party recomputable by construction. AI-control evaluations [12] study protocols that remain safe when the agent itself is untrusted. The referee is a run-time example of that stance, since nothing the agent asserts is load-bearing and trust is a computed property of graded receipts. Calibration studies show models carry usable self-knowledge about factual claims [17]. The COVERAGE AUDIT addresses a complementary failure: confident conclusions resting on rules the record never exercised. Goal-drift evaluations [3] document the adjacent failure family in long-horizon agents, and long-context degradation [24] is one mechanism by which prompt-level steering decays.

Tamper-evident records. The journal’s construction is standard cryptographic practice, hash-linked append-only logs with external anchoring, in the lineage of secure digital timestamping [13] and forensic audit logs [29], applied here to agent runs.

10 Limitations

Every cell is $n=1$ by protocol, and run-to-run variance is real and uninstrumented here. All our runs use one backbone (Claude Opus 5), so architecture-vs-backbone is answered only by same-backbone comparators, not by swaps. Comparisons use each competitor’s own published card (Prime Agent’s median, with best-of- N regimes labeled as such). PRO-LONG ran on a stronger backbone, and its column is context, not controlled comparison. Our public scorecards are verification replays of recorded action sequences, with provenance stated (footnote 2). They confirm outcomes and counts, not live reasoning. RHAE is reverse-derived, though validated 25/25 exactly and independently confirmed by the server’s own scoring of our set. A human-expert reference of 95.4 circulates for this benchmark. It is Prime Intellect’s derivation, which we could not verify on arcprize.org, so we attribute it and hang no claim on it. If52 is unsolved and reported as the standing falsifier. The finding in §5 has $n=6$ across five games in one benchmark family, and its generality beyond ARC-AGI-3 is conjecture until the planned cross-domain runs. The Factorio and OOLONG results are first sweeps rather than scored benchmarks, three calibration tasks on Factorio and one corpus per length rung on OOLONG, each cell $n=1$ and each an early alpha evaluation, not the wide runs their framing would ultimately require. The harness itself is not released. The verification surface is public and MIT-licensed (footnote 1), so third parties can verify every claim in this paper without trusting us.

11 Conclusion

This paper set out to make an agent reason through an unfamiliar world, learn it from interaction, and keep what it learns, under one discipline rather than a pile of prompt rules. A single rule, no action without a prediction graded in code, forces the agent to build a world model under scarcity, and on ARC-AGI-3 the gated agent reaches the frontier tier under hard caps at a measured 8.0% exploration overhead and approximately zero capability cost over completed games. The auditable journal is a by-product of that same discipline, and it is not only accountability. The same journal that makes the runs verifiable is what let the COVERAGE AUDIT break six impossibility proofs the agent itself had authored, converting five lost games into wins. When an agent reports that something is impossible, the productive question is which rule of that proof was ever actually exercised. The same harness, unchanged, ported to two further worlds. On Factorio it fail-closes the sanctioned-interface bypass a published agent fell to and wins its calibration tasks without needing it, and on OOLONG’s long-context ladder an early sweep already sits in the band of mature general harnesses.

Reproducibility. Set score, per-game outcomes, and per-level action counts are independently checkable on public card 702ccd4f (footnote 2), and every competitor number traces to the cards in footnote 5. The RHAE scorer with baselines, the journal-format specification, the claim-grammar specification, the standalone audit/replay verifier, and a complete sample journal are public and MIT-licensed at <https://github.com/arjmandi/assay-verify>. The project page is <https://arjmandi.github.io/assay-site/>.

<https://arcprize.org/scorecards/2af780b4-f2a1-43e9-a794-b23da3cd3f9f>. PRO-LONG’s 25 per-game card links are published at its project page.

References

- [1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. arXiv:1606.06565, 2016.
- [2] ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. arXiv:2603.24621, 2026.
- [3] Rauno Arike, Elizabeth Donoway, Henning Bartsch, and Marius Hobbhahn. Technical report: Evaluating goal drift in language model agents. arXiv:2505.02709, 2025.
- [4] Mohsen Arjmandi. The LLM proposes, the executive disposes: A self-verifying agent instrument that dissociates commitment drift from binding drift in long-horizon agents. arXiv:2608.04066, 2026.
- [5] Mohsen Arjmandi. Sensi: Learn one thing at a time — curriculum-based test-time learning for LLM game agents. arXiv:2603.17683, 2026.
- [6] Amanda Bertsch, Adithya Pratapa, Teruko Mitamura, Graham Neubig, and Matthew R. Gormley. Oolong: Evaluating long context reasoning and aggregation capabilities. arXiv:2511.02817, 2025.
- [7] Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Bluemke, Nitarshan Rajkumar, David Krueger, Noam Kolt, Lennart Heim, and Markus Anderljung. Visibility into AI agents. arXiv:2401.13138, 2024.
- [8] François Chollet. On the measure of intelligence. arXiv:1911.01547, 2019.
- [9] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2024: Technical report. arXiv:2412.04604, 2024.
- [10] Alexis Fox, Junlin Wang, Paul Rosu, and Bhuwan Dhingra. PRO-LONG: Programmatic memory enables long-horizon reasoning. arXiv:2607.20064, 2026.
- [11] Paul Furgale et al. NVIDIA-labs OO agents: Native Python object-oriented agents. arXiv:2607.20709, 2026.
- [12] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. arXiv:2312.06942, 2023.
- [13] Stuart Haber and W. Scott Stornetta. How to time-stamp a digital document. *Journal of Cryptology*, 3(2): 99–111, 1991.
- [14] Liew Keong Han. Explore before you solve: The speed–depth trade-off in epistemic agents for ARC-AGI-3. arXiv:2605.25931, 2026.
- [15] Jack Hopkins, Mart Bakler, and Akbir Khan. Factorio learning environment. arXiv:2503.09617, 2025.
- [16] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. arXiv:2310.01798, 2023.
- [17] Saurav Kadavath, Tom Conerly, Amanda Askell, et al. Language models (mostly) know what they know. arXiv:2207.05221, 2022.
- [18] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. LLMs can’t plan, but can help planning in LLM-modulo frameworks. arXiv:2402.01817, 2024.
- [19] Sayash Kapoor, Benedikt Stroebel, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. arXiv:2407.01502, 2024.
- [20] Megan Kinniment, Lucas Jun Koba Sato, Haoxing Du, Brian Goodrich, Max Hasin, Lawrence Chan, Luke Harold Miles, Tao R. Lin, Hjalmar Wijk, Joel Burget, Aaron Ho, Elizabeth Barnes, and Paul Christiano. Evaluating language-model agents on realistic autonomous tasks. arXiv:2312.11671, 2023.
- [21] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: the flip side of AI ingenuity. DeepMind blog, <https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>, 2020.
- [22] Jens Lehmann, Andrei Aioanei, and Sahar Vahdati. Tycho: Active abstraction with programmatic world models for ARC-AGI-3. arXiv:2607.28287, 2026.
- [23] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. arXiv:2305.20050, 2023.
- [24] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. arXiv:2307.03172, 2023.
- [25] Silen Naihin, David Atkinson, Marc Green, Merwane Hamadi, Craig Swift, Douglas Schonholtz, Adam Tautman Kalai, and David Bau. Testing language model agents safely in the wild. arXiv:2311.10538, 2023.
- [26] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. arXiv:2201.03544, 2022.

- [27] Prime Intellect. Prime agent. Blog post, <https://www.primeintellect.ai/blog/prime-agent>, 2026.
- [28] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. arXiv:2309.15817, 2023.
- [29] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999.
- [30] Yonadav Shavit, Sandhini Agarwal, Miles Brundage, et al. Practices for governing agentic AI systems. OpenAI white paper, <https://openai.com/index/practices-for-governing-agentic-ai-systems/>, 2023.
- [31] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. arXiv:2303.11366, 2023.
- [32] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krashennnikov, and David Krueger. Defining and characterizing reward hacking. arXiv:2209.13085, 2022.
- [33] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. arXiv:2402.08115, 2024.
- [34] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. arXiv:2211.14275, 2022.
- [35] UK AI Safety Institute. Inspect: An open-source framework for large language model evaluations. Software, https://github.com/UKGovernmentBEIS/inspect_ai, 2024.
- [36] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv:2305.16291, 2023.
- [37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. arXiv:2210.03629, 2022.